

# Enterprise IT Lab

Project Documentation

---

*Ibnousoufyane Ayman*

June 2026

[github.com/Ayman6594/AD-Project](https://github.com/Ayman6594/AD-Project)

## **This document covers:**

- Active Directory — Domain Controller, DNS, DHCP, GPOs
- Client-01 — Windows 11 domain join and GPO verification
- Monitor-01 — Prometheus, Node Exporter, Grafana live dashboard
- Graylog — Centralized Windows log collection via NXLog

## Lab Architecture

A fully functional enterprise IT environment running on VMware Workstation. Four virtual machines communicate over a Host-Only internal network with internet access via NAT. The lab replicates real-world corporate infrastructure found in companies across Morocco and internationally.

### Virtual Machines and Network

| VM         | OS                  | IP Address         | RAM  | Role                         |
|------------|---------------------|--------------------|------|------------------------------|
| DC01       | Windows Server 2022 | 192.168.1.10       | 4 GB | Domain Controller, DNS, DHCP |
| Client-01  | Windows 11          | 192.168.1.x (DHCP) | 4 GB | Domain workstation           |
| Monitor-01 | Ubuntu 24.04 LTS    | 192.168.1.101      | 4 GB | Prometheus, Grafana, Graylog |

### Services and Access URLs

| Service       | URL / Address                     | Credentials         |
|---------------|-----------------------------------|---------------------|
| Grafana       | http://192.168.1.101:3000         | admin / User@123456 |
| Prometheus    | http://192.168.1.101:9090         | No auth             |
| Node Exporter | http://192.168.1.101:9100/metrics | No auth             |
| Graylog       | http://192.168.1.101:9000         | admin / admin       |
| DC01 RDP      | 192.168.1.10                      | IB\Administrateur   |
| Domain Login  | IB\username                       | User@123456         |

## 1. Active Directory

Windows Server 2022 was installed and promoted as the Domain Controller for the IB.local forest. The entire configuration was automated using a single PowerShell script — from OU structure to GPO deployment — demonstrating infrastructure-as-code in a Windows environment.

### Domain Information

- Domain name: IB.local
- NetBIOS name: IB
- Forest and domain functional level: Windows Server 2016
- Domain Controller: DC01 — 192.168.1.10
- DNS: integrated with AD, pointing to 127.0.0.1

### Automation Script

A PowerShell script (AD-Setup.ps1) was written to deploy the full AD configuration automatically. It runs in under 5 minutes and creates the complete environment from scratch.

```
Set-ExecutionPolicy Unrestricted -Force  
.\AD-Setup.ps1
```

### What the script creates:

- 6 Organizational Units: IB-Company, IT, Finance, Management, Users, Computers, Service-Accounts
- 4 Security Groups: GRP-IT, GRP-Finance, GRP-Management, GRP-AllUsers
- 5 Domain Users assigned to their respective OUs and groups
- DHCP Server installed, authorized, and scope configured
- 6 GPOs created and linked to their target OUs

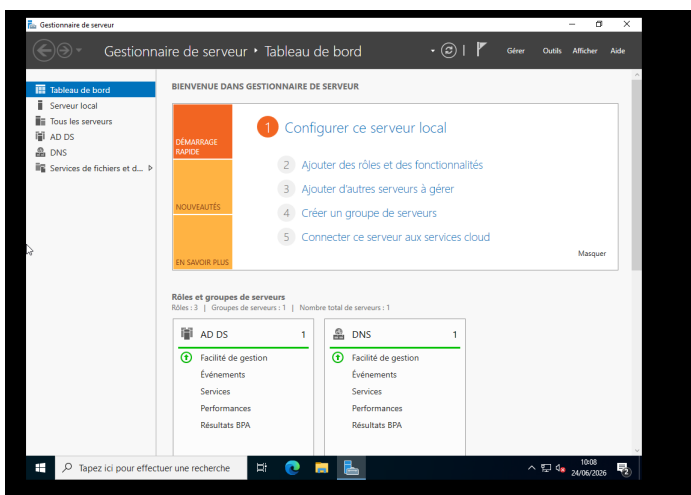
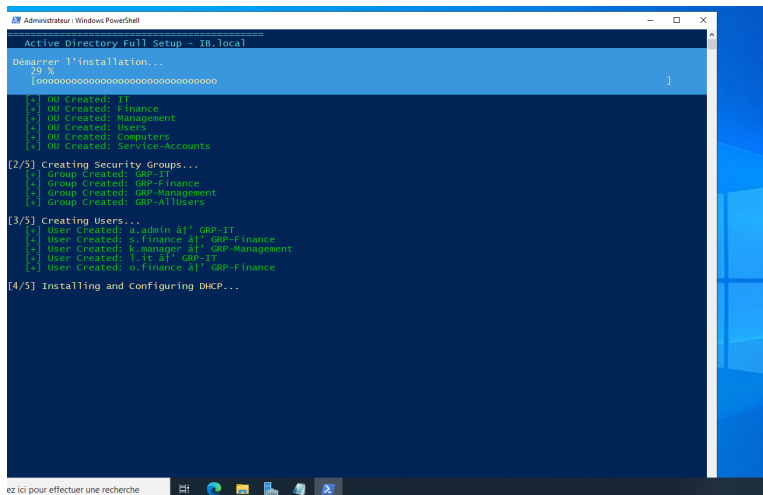


Figure 1 — Server Manager showing AD DS and DNS services active and green



```
Administrateur : Windows PowerShell
Active Directory Full Setup - IB.local

Demarrer l'installation...
[1/5] Creating Organizational Units...
[+] OU Created: IT
[+] OU Created: Finance
[+] OU Created: Management
[+] OU Created: Users
[+] OU Created: Computers
[+] OU Created: Service-Accounts

[2/5] Creating Security Groups...
[+] Group Created: GRP-IT
[+] Group Created: GRP-Finance
[+] Group Created: GRP-Management
[+] Group Created: GRP-AllUsers

[3/5] Creating Users...
[+] User Created: s.admin@IB.local GRP-IT
[+] User Created: l.manager@IB.local GRP-Management
[+] User Created: l.it@IB.local GRP-IT
[+] User Created: o.finance@IB.local GRP-Finance

[4/5] Installing and Configuring DHCP...
```

Figure 2 — PowerShell script running: OUs, Groups and Users created automatically

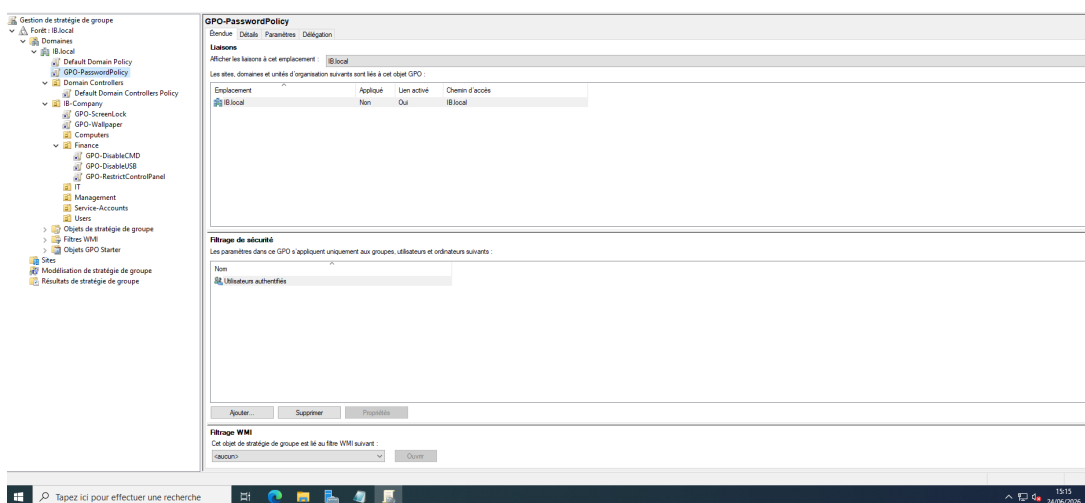


Figure 3 — Group Policy Management Console showing all 6 GPOs linked to their OUs

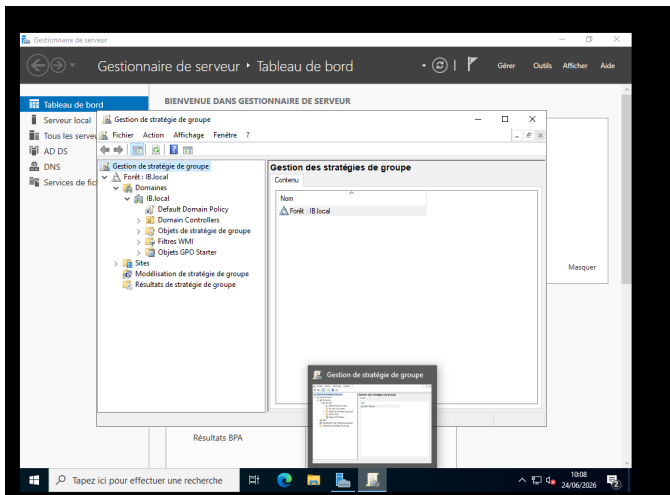


Figure 4 — Full OU and GPO tree structure in IB.local domain

## Users Created

- a.admin (Ayman Admin) — IT — GRP-IT — IT Administrator
- l.it (Leila IT) — IT — GRP-IT — IT Support
- s.finance (Sara Finance) — Finance — GRP-Finance — Financial Analyst
- o.finance (Omar Finance) — Finance — GRP-Finance — Accountant
- k.manager (Karim Manager) — Management — GRP-Management — Department Manager

## GPOs Deployed

- GPO-PasswordPolicy → IB.local: strong password + 90-day expiry
- GPO-ScreenLock → IB-Company: screen locks after 10 minutes
- GPO-Wallpaper → IB-Company: company wallpaper on all machines
- GPO-DisableUSB → Finance: USB storage blocked
- GPO-RestrictControlPanel → Finance: Control Panel inaccessible
- GPO-DisableCMD → Finance: CMD and PowerShell blocked

## GitHub

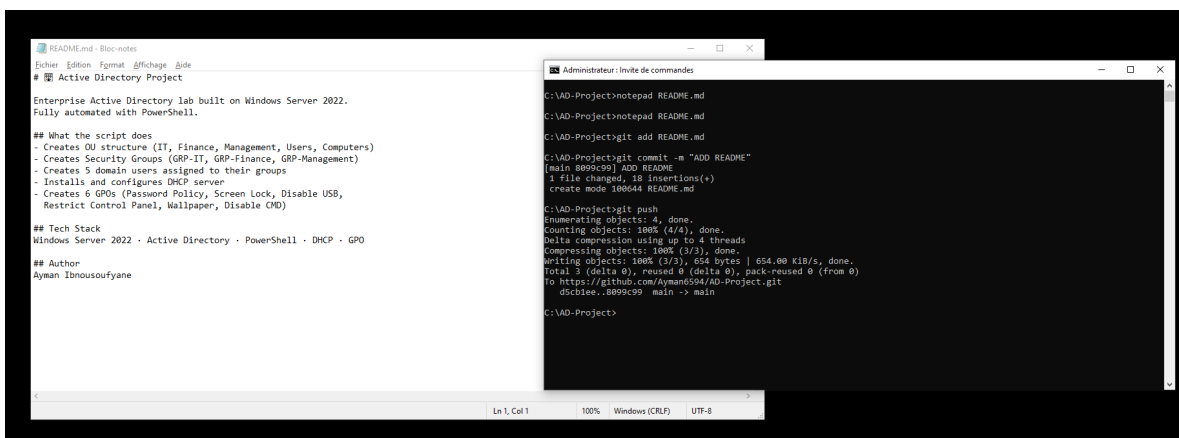


Figure 5 — GitHub repository AD-Project with PowerShell script and README

## 2. Client-01 — Domain Workstation

A Windows 11 virtual machine was joined to the IB.local domain. Domain users can log in from the workstation and all GPOs apply automatically at login — exactly as in a real corporate environment. The machine was given a Host-Only network adapter so it can reach DC01 at 192.168.1.10.

### Domain Join Steps

- Added Host-Only network adapter in VMware settings
- Set DNS to 192.168.1.10 (DC01) on the Host-Only adapter
- System Properties → Change → Domain: IB.local
- Authenticated with IB\Administrateur — restart
- Logged in with IB\\*.admin and IB\\*.finance — confirmed working

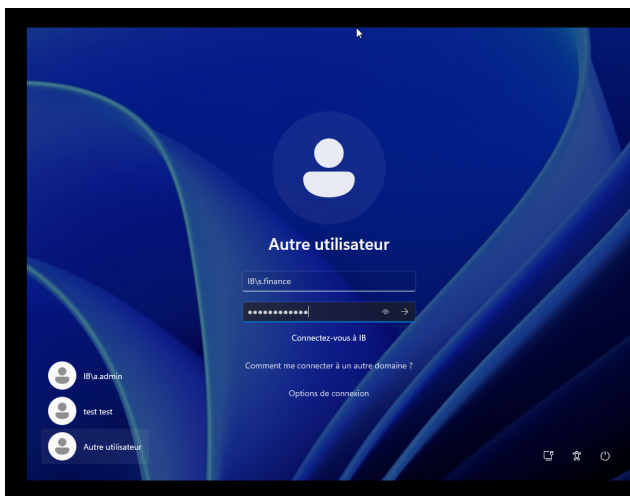


Figure 6 — Windows 11 login screen: IB domain users visible — IB\\*.admin and IB\\*.finance

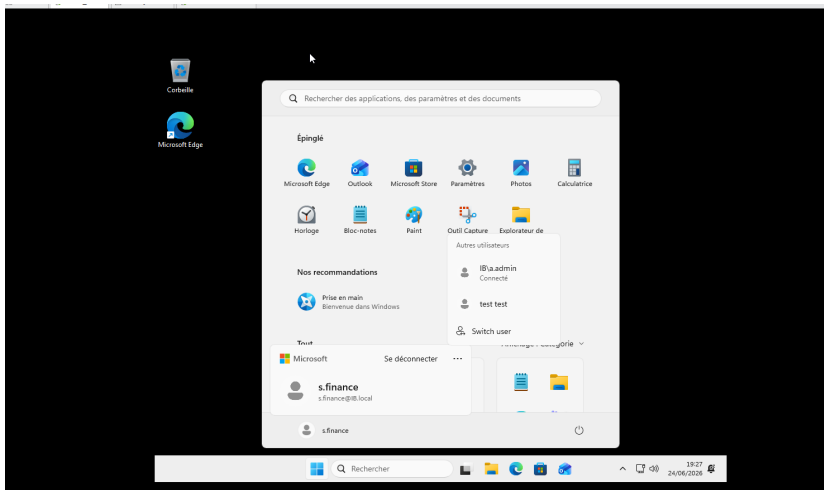


Figure 7 — Start menu logged in as s.finance@IB.local — domain account confirmed

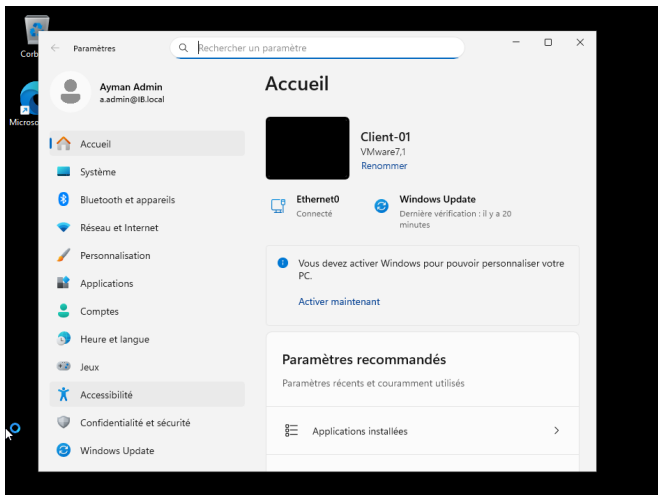


Figure 8 — System Settings showing a.admin@IB.local on Client-01 (machine: Client-01)

## GPO Verification

After logging in as s.finance (Finance group), all Finance-targeted GPOs were tested and confirmed working on Client-01.

- CMD opened → immediately shows "disabled by your administrator" check
- Control Panel → "Operation cancelled due to restrictions on this computer" check
- USB storage device → blocked by the system check

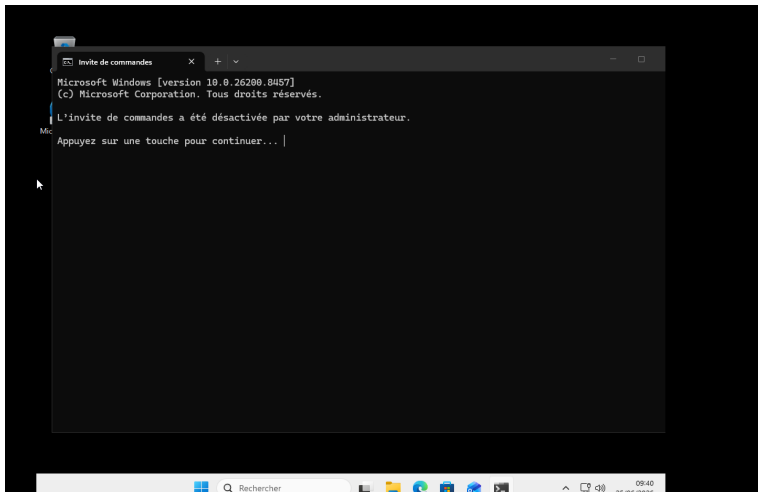


Figure 9 — CMD blocked: 'The command prompt has been disabled by your administrator'

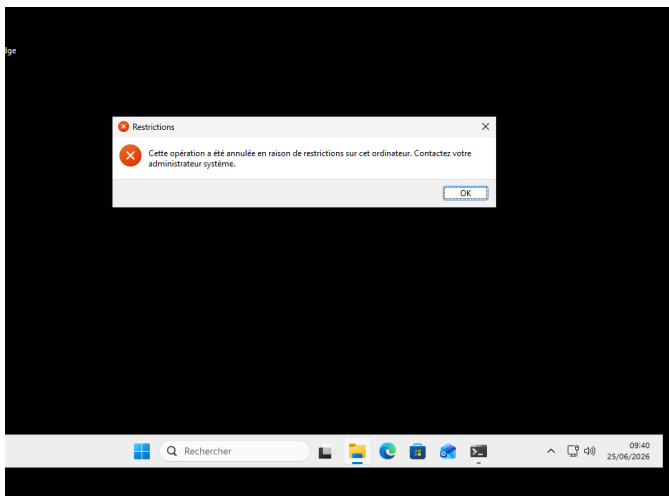


Figure 10 — Control Panel blocked: 'Operation cancelled due to restrictions'



### 3. Monitoring — Prometheus + Grafana

An Ubuntu 24.04 LTS server (Monitor-01) was deployed as a dedicated monitoring node. Using a single bash script, the full monitoring stack was installed and configured: Prometheus collects metrics every 15 seconds, Node Exporter exposes Linux system stats, and Grafana displays everything in a live dashboard accessible from any machine in the lab.

#### Ubuntu Server Installation

- Ubuntu 24.04 LTS ISO downloaded from [ubuntu.com/server](https://ubuntu.com/server)
- New VM created in VMware: 4GB RAM, 60GB disk, NAT + Host-Only adapters
- Static IP set via netplan: 192.168.1.101 on ens34 (Host-Only)
- Ping to DC01 (192.168.1.10) confirmed — network working

#### Netplan configuration applied

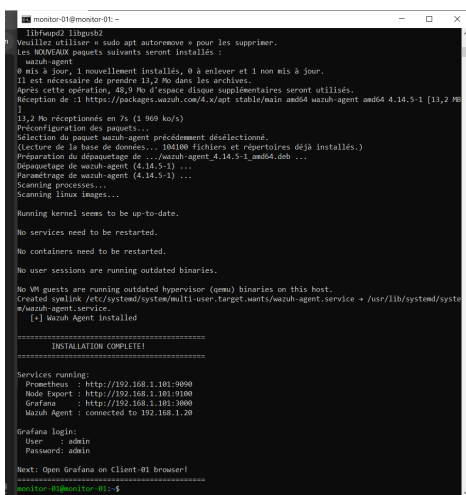
```
sudo nano /etc/netplan/00-installer-config.yaml
sudo chmod 600 /etc/netplan/00-installer-config.yaml
sudo netplan apply
ping 192.168.1.10
```

#### Installation Script

A bash script (install-monitoring.sh) installs and configures everything automatically. It creates system users, downloads binaries, writes systemd service files, and starts all services.

```
chmod +x ~/install-monitoring.sh
sudo ~/install-monitoring.sh
```

- Prometheus installed at /usr/local/bin/prometheus — port 9090
- Node Exporter installed at /usr/local/bin/node\_exporter — port 9100
- Grafana installed via official apt repository — port 3000
- Windows Exporter installed on DC01 via PowerShell — port 9182



```
monitor-01@monitor-01:~$ ./install-monitoring.sh
libfuse2 libfuse3
Veuillez utiliser « sudo apt autoremove » pour les supprimer.
Les WAZUH paquets suivants seront installés :
  wazuh-agent
0 MiB à télécharger, 1 nouvellement installés, 0 à enlever et 1 non mis à jour.
Il est nécessaire de prendre 13,2 Mo dans les archives.
Après cette opération, 48,9 Mo d'espace disque supplémentaires seront utilisés.
Réception de « https://packages.wazuh.com/4.x/apt/stable/main amd64 wazuh-agent amd64 4.14.5-1 [13,2 Mo] »
13,2 Mo réceptionnés en 7s (1 969 ko/s)
Préconfiguration des paquets...
Sélection du paquet wazuh-agent précédemment désélectionné.
(Lecture de la base de données... 18180 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../wazuh-agent_4.14.5-1_amd64.deb ...
Dépaquetage de wazuh-agent (4.14.5-1) ...
Paramétrage de wazuh-agent (4.14.5-1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service → /usr/lib/systemd/system/wazuh-agent.service.
[*] Wazuh Agent installed

=====
INSTALLATION COMPLETE!
=====

Services running:
Prometheus : http://192.168.1.101:9090
Node Export : http://192.168.1.101:9100
Grafana     : http://192.168.1.101:3000
Wazuh Agent : connected to 192.168.1.20

Grafana login:
User      : admin
Password : admin

Next: Open Grafana on Client-01 browser!

monitor-01@monitor-01:~$
```

Figure 11 — Bash script output: all components installed and services started

## Grafana Configuration

- Opened at <http://192.168.1.101:3000> from Client-01 browser
- Default login: admin / admin — changed password on first login
- Data source added: Prometheus at <http://localhost:9090>
- Dashboard imported from Grafana.com: Node Exporter Dashboard
- Instance selected: localhost:9100 — Job: node-exporter

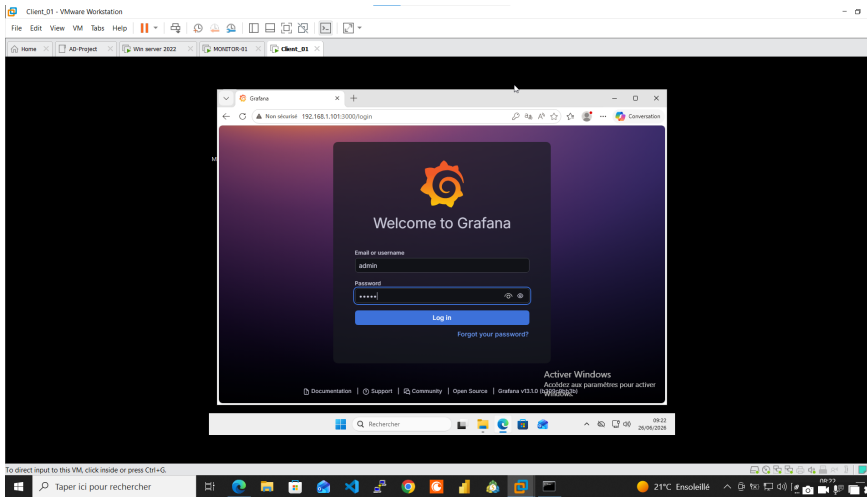


Figure 12 — Grafana login page accessed from Client-01 at <http://192.168.1.101:3000>

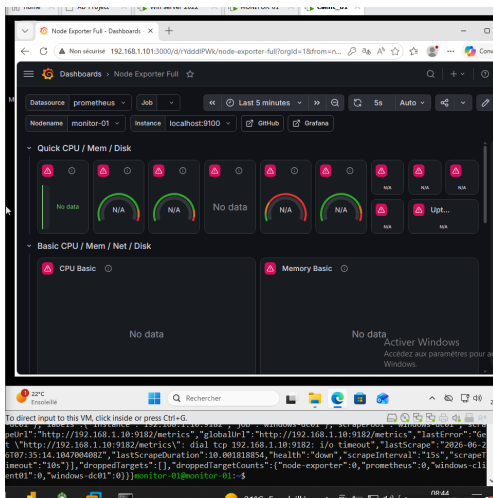


Figure 13 — Dashboard live: CPU 4.9%, RAM 30.9%, Disk 4.9GB used, instance monitor-01

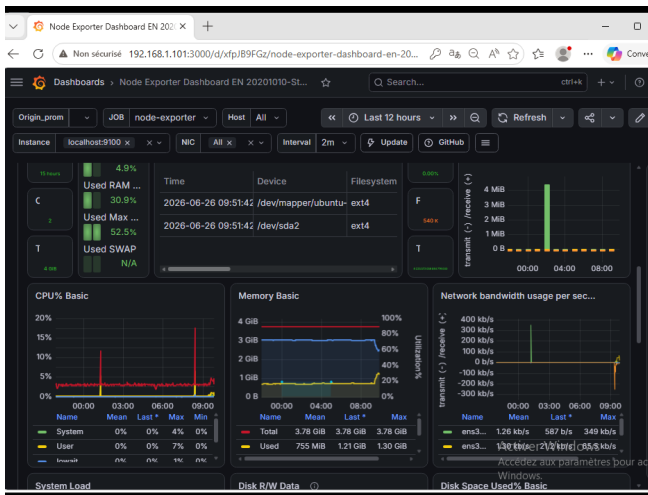


Figure 14 — Full dashboard: CPU Basic, Memory Basic, Network bandwidth, Disk R/W in real-time

## Metrics collected every 15 seconds

- CPU usage per core — idle, user, system, iowait modes
- RAM total 3.78 GiB — used 1.21 GiB — cached
- Disk I/O read/write — /dev/mapper/ubuntu ext4 and /dev/sda2
- Network bandwidth — ens33 (NAT) and ens34 (Host-Only)
- System load average 1m / 5m / 15m
- Filesystem usage percentage

```
lines 1-20/20 (END)
monitor-01@monitor-01:~$ curl http://localhost:9090/api/v1/targets
curl: (6) Could not resolve host: localhost
monitor-01@monitor-01:~$ curl http://localhost:9090/api/v1/targets
{"status": "Success", "data": {"activeTargets": [{"discoveredLabels": {"__address__": "localhost:9100", "__metrics_path__": "/metrics", "__scheme__": "http", "__scrape_interval__": "15s", "__scrape_timeout__": "10s", "job": "node-exporter"}, "labels": {"instance": "localhost:9100", "job": "node-exporter"}, "scrapePool": "node-exporter", "scrapeUrl": "http://localhost:9100/metrics", "globalUrl": "http://monitor-01:9100/metrics", "lastError": "", "lastScrape": "2026-06-26T07:35:31.980943732Z", "lastScrapeDuration": 0.018219046, "health": "up", "scrapeInterval": "15s", "scrapeTimeout": "10s"}, {"discoveredLabels": {"__address__": "localhost:9090", "__metrics_path__": "/metrics", "__scheme__": "http", "__scrape_interval__": "15s", "__scrape_timeout__": "10s", "job": "prometheus"}, "labels": {"instance": "localhost:9090", "job": "prometheus"}, "scrapePool": "prometheus", "scrapeUrl": "http://localhost:9090/metrics", "globalUrl": "http://monitor-01:9090/metrics", "lastError": "", "lastScrape": "2026-06-26T07:35:31.22869942Z", "lastScrapeDuration": 0.005261322, "health": "up", "scrapeInterval": "15s", "scrapeTimeout": "10s"}, {"discoveredLabels": {"__address__": "192.168.1.101:9182", "__metrics_path__": "/metrics", "__scheme__": "http", "__scrape_interval__": "15s", "__scrape_timeout__": "10s", "job": "windows-client01"}, "labels": {"instance": "192.168.1.101:9182", "job": "windows-client01"}, "scrapePool": "windows-client01", "scrapeUrl": "http://192.168.1.101:9182/metrics", "globalUrl": "http://192.168.1.101:9182/metrics", "lastError": "Get \"http://192.168.1.101:9182/metrics\": dial tcp 192.168.1.101:9182: connect: connection refused", "lastScrape": "2026-06-26T07:35:34.518274492Z", "lastScrapeDuration": 0.000657902, "health": "down", "scrapeInterval": "15s", "scrapeTimeout": "10s"}, {"discoveredLabels": {"__address__": "192.168.1.10:9182", "__metrics_path__": "/metrics", "__scheme__": "http", "__scrape_interval__": "15s", "__scrape_timeout__": "10s", "job": "windows-dc01"}, "labels": {"instance": "192.168.1.10:9182", "job": "windows-dc01"}, "scrapePool": "windows-dc01", "scrapeUrl": "http://192.168.1.10:9182/metrics", "globalUrl": "http://192.168.1.10:9182/metrics", "lastError": "Get \"http://192.168.1.10:9182/metrics\": dial tcp 192.168.1.10:9182: i/o timeout", "lastScrape": "2026-06-26T07:35:14.104700408Z", "lastScrapeDuration": 0.001818854, "health": "down", "scrapeInterval": "15s", "scrapeTimeout": "10s"}], "droppedTargetCounts": {"node-exporter": 0, "prometheus": 0, "windows-client01": 0, "windows-dc01": 0}}}
monitor-01@monitor-01:~$
```

Figure 15 — Prometheus targets: node-exporter UP, windows exporters configured

## 4. Graylog — Centralized Log Management

After encountering severe compatibility issues installing Wazuh SIEM on Ubuntu 24.04 (both the native package and the Docker deployment crashed repeatedly due to API and SSL certificate bugs), the decision was made to switch to Graylog — a simpler and equally professional centralized log management platform widely used in real companies. Graylog collects, indexes, and makes searchable all Windows Event Logs from DC01 and Client-01, giving full visibility across the domain from a single web interface.

### Why Graylog over Wazuh

- Wazuh Manager API repeatedly crashed on Ubuntu 24.04 (process exits right after "Listening")
- Wazuh Docker deployment failed on certificate mounting (EISDIR errors)
- Graylog needs only 3 containers (MongoDB, OpenSearch, Graylog) vs 3+ for Wazuh with more SSL complexity
- Graylog deployed cleanly on the first attempt with Docker Compose

### Architecture

| Component  | Role                                      | Port                         |
|------------|-------------------------------------------|------------------------------|
| MongoDB    | Stores Graylog configuration and metadata | 27017 (internal)             |
| OpenSearch | Indexes and stores all log messages       | 9200 (internal)              |
| Graylog    | Web UI, REST API, log ingestion (GELF)    | 9000 (web), 12201/udp (GELF) |

### Docker Compose Deployment

A docker-compose.yml file defines all three services. Default login credentials are admin / admin (root password hash pre-configured in the environment variables).

```
mkdir -p ~/graylog && cd ~/graylog
nano docker-compose.yml
# (paste the 3-service stack: mongodb, opensearch, graylog)
sudo docker-compose up -d
```

### Key environment variables for the graylog service:

```
GRAYLOG_ROOT_PASSWORD_SHA2=8c6976e5b541... (sha256 of "admin")
GRAYLOG_HTTP_EXTERNAL_URI=http://192.168.1.101:9000/
GRAYLOG_ELASTICSEARCH_HOSTS=http://opensearch:9200
GRAYLOG_MONGODB_URI=mongodb://mongodb:27017/graylog
```

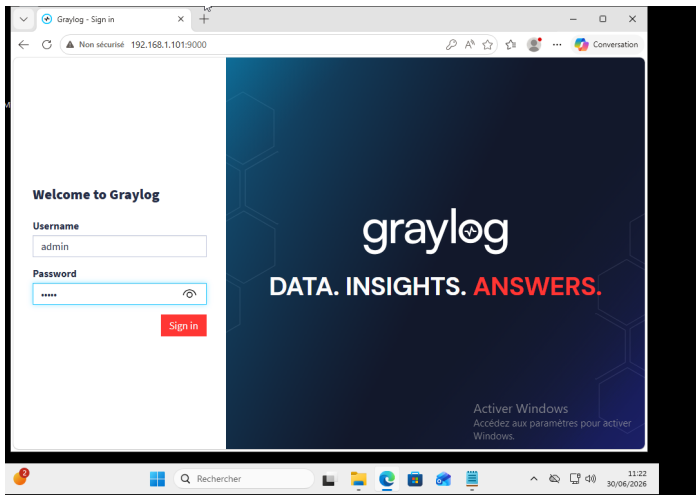


Figure 16 — docker-compose up -d: all 3 containers (mongodb, opensearch, graylog) created successfully

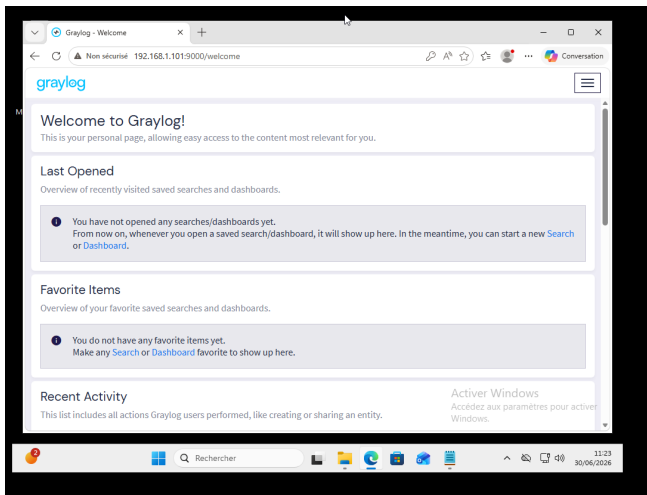


Figure 17 — Graylog Welcome page after first login (admin/admin) from Client-01

## Creating the GELF Input

A GELF UDP input was created in Graylog to receive log messages on port 12201. This is the endpoint that NXLog (installed on the Windows machines) forwards Windows Event Logs to.

- System → Inputs → select "GELF UDP" → Launch new input
- Title: Windows-Logs | Bind address: 0.0.0.0 | Port: 12201

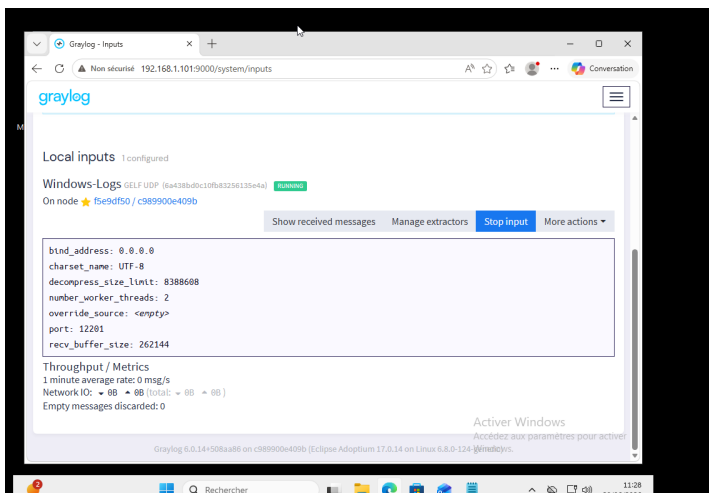


Figure 18 — Windows-Logs GELF UDP input RUNNING on port 12201

## Installing NXLog on DC01 and Client-01

NXLog Community Edition was installed on both Windows machines to read the Windows Event Log (Application, System, Security channels) and forward each event to Graylog in GELF format over UDP.

```
notepad "C:\Program Files\nxlog\conf\nxlog.conf"
```

### Key configuration block (same on both machines, Host set to Monitor-01):

```
Module im_msvistalog
Query ...Application/System/Security...

Module om_udp
Host 192.168.1.101
Port 12201
OutputType GELF

Restart-Service nxlog
Get-Service nxlog # Status: Running
```

## Automated GPO Deployment (DC01 to Client-01)

To make the deployment repeatable and demonstrate enterprise-style software distribution, a GPO was configured to push the NXLog MSI to all domain computers automatically via Computer Configuration → Software Installation. A shared folder hosts the MSI and NTFS/share permissions were granted to Domain Computers so machine accounts can read it.

```
New-Item -Path "C:\Software" -ItemType Directory -Force
New-SmbShare -Name "Software" -Path "C:\Software" -FullAccess "S-1-1-0"
icacls C:\Software /grant "Utilisateurs authentifiés:(OI)(CI)F"
icacls C:\Software /grant "Ordinateurs du domaine:(OI)(CI)F"
New-GPO -Name "GPO-Deploy-NXLog"
New-GroupPolicyLink -Name "GPO-Deploy-NXLog" -Target "OU=Computers,OU=IB-Company,DC=IB,DC=local"
```

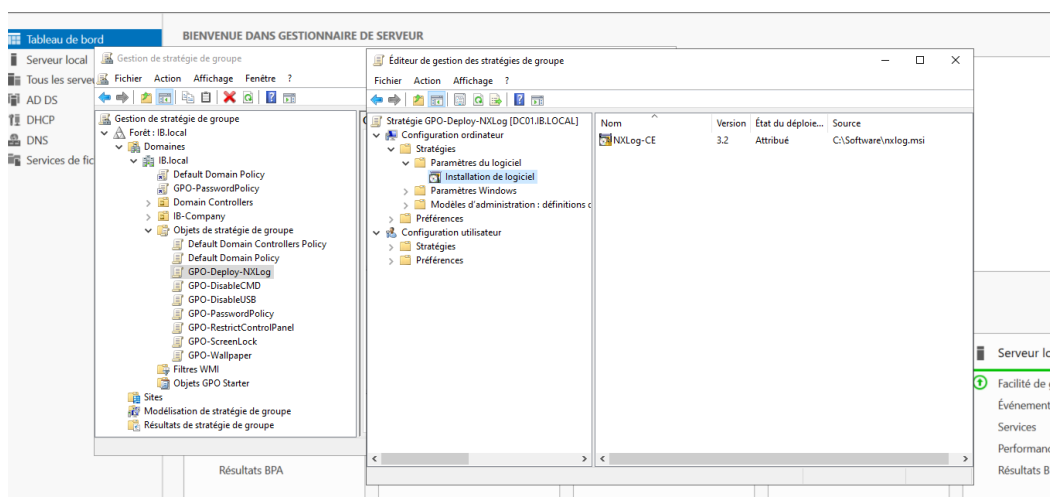


Figure 19 — Software Installation package NXLog-CE assigned via GPO-Deploy-NXLog, source \\DC01\Software\nxlog.msi

## Results — Logs Arriving in Real Time

With NXLog running on both DC01 and Client-01, Windows Security and System events (logons, logoffs, special privilege assignments, service state changes) now stream into Graylog continuously and are fully searchable.

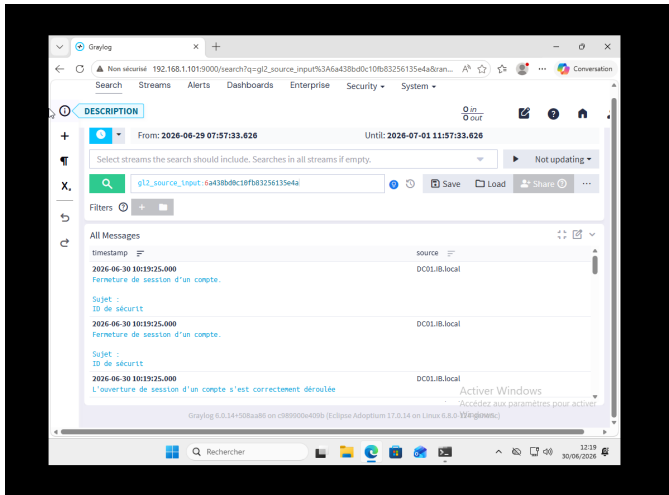


Figure 20 — First logs arriving from DC01.IB.local: account logon/logoff events, security IDs

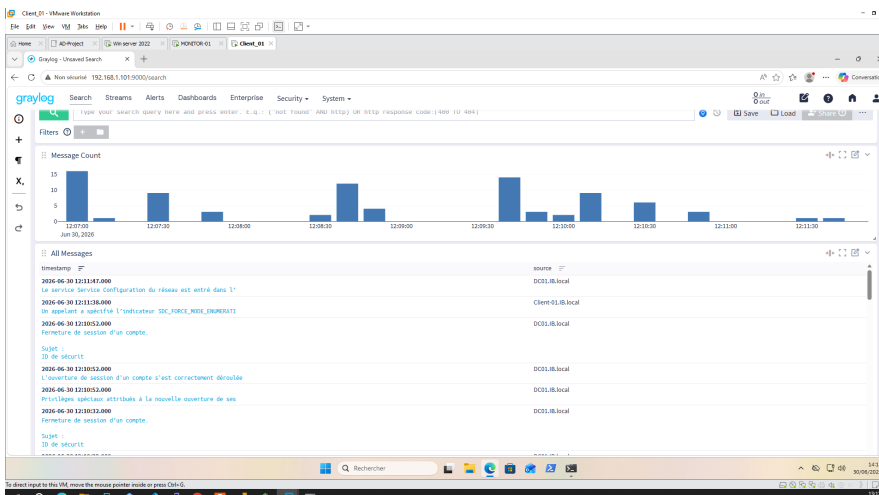


Figure 21 — Logs from BOTH DC01.IB.local and Client-01.IB.local in the same search view, with live message-count graph

## Log sources confirmed working:

- DC01.IB.local — account logon/logoff, special privileges, service configuration events
- Client-01.IB.local — system events, network configuration changes
- Message count graph showing continuous activity, updating live
- Full-text search across all collected fields



## What's Next

---

The lab continues to grow. Each upcoming step adds a new enterprise layer on top of what is already running.

### Graylog Alerts

- Configure alert conditions on failed logon attempts (multiple Event ID 4625)
- Email or webhook notifications on suspicious activity
- Dashboards with widgets for top users, top event types, login trends

### VPN + AD Integration

- Connect the SSL VPN (ocserv) to Active Directory via LDAP
- Domain users authenticate to VPN with their AD credentials
- Same principle as FortiClient with LDAP/AD integration

#### Enterprise IT Lab — Ibnousoufyane Ayman

June 2026 | VMware Workstation | Windows Server 2022 · Windows 11 · Ubuntu 24.04

Stack: Active Directory · GPO · Prometheus · Grafana · Graylog · NXLog

Repository: [github.com/Ayman6594/AD-Project](https://github.com/Ayman6594/AD-Project)